

Q1. Which address(es) should be allowed to mint/burn the LP tokens?

Answer:

- Only the DEX contract address should be allowed to mint and burn LP tokens.
- This is enforced in LPToken.sol using `require(msg.sender == dex, "Only DEX can mint/burn")`
- In the implementation:
 - LP tokens are minted only inside `addLiquidity()`
 - LP tokens are burned only inside `removeLiquidity()`
- If arbitrary users could mint LP tokens:
 - they could claim pool ownership without depositing assets
 - reserves and ownership accounting would diverge

Q2. In what way do DEXs level the playing ground between a powerful and resourceful trader (HFT/institutional investor) and a lower resource trader (retail investors, like you and me!)?

Answer:

- DEXs remove the need for intermediaries and order books, replacing them with deterministic pricing based on reserves
- Price is derived from reserves (`reserveA`, `reserveB`)
- Swap execution is formula-driven, not negotiation-driven
- Retail users and large traders both:
 - See the same pool
 - Get the same pricing curve
 - Execute under identical rules

Q3. Suppose there are many transaction requests to the DEX sitting in a miner's mempool. How can the miner take undue advantage of this information? Is it possible to make the DEX robust against it?

Answer:

- When transactions sit in the mempool, miners can see pending swaps before they are executed.
- Miners can frontrun or sandwich trades by seeing pending transactions.
- Example:
 - user submits `swapAForB(amount)`
 - miner sees it will move price
 - miner executes swap first, gets better rate
 - user executes later, worse rate due to updated reserves
- It is difficult to fully prevent on public mempools
- Mitigation techniques include private RPCs, slippage tolerance (user-defined limits) etc. (not implemented in this basic version)

Q4. We have left out a very important dimension on the feasibility of this smart contract- the gas fees! Every function call needs gas. How does gas fees influence economic viability of the entire DEX and arbitrage?

Answer:

- Gas fees make very small trades and small arbitrage opportunities unprofitable.
- Every interaction with the DEX involves gas, every interaction with the DEX involves gas!
- In Swaps:
 - small trades may become uneconomical
 - small trades may become uneconomical
- In Arbitrage:
 - `uint256 p1 = _tryAtoBthenBtoA(dex1, dex2, amountA);`
 - `uint256 p2 = _tryAtoBthenBtoA(dex2, dex1, amountA);`
 - `require(profit >= MIN_PROFIT, "No profitable arbitrage");`
 - We enforce a MIN_PROFIT threshold to ensure the profit covers gas costs.

Q5. Could gas fees lead to undue advantages to some transactors over others? How?

Answer:

- Yes. Users who can pay higher gas prices (priority fees) can frontrun others. Large players have an advantage in competitive situations.
- Higher-paying users can:
 - prioritize their transactions in the block
 - outbid others in congested conditions
- Retail users often lose on execution timing

Q6. What are the various ways to minimize slippage (as defined in 4.2) in a swap?

- Slippage arises due to the constant product formula and reserve impact.
- Increase overall liquidity in the pool
- Make smaller trade sizes
- Arbitrage alignment
- Reject trades beyond acceptable price

Q7. Having defined what slippage, plot how slippage varies with the “trade lot fraction” for a constant product AMM. Trade lot fraction is the ratio of the amount of token X deposited in a swap, to the amount of X in the reserves just before the swap.

Answer:

- $\text{trade_fraction} = \text{amountIn} / \text{reserve}$
- From AMM behavior:
 - When fraction is small, slippage is negligible or approx linear
 - As fraction increases, slippage increases non-linearly
- As amountIn grows:

- denominator increases
- marginal output decreases